MySQL

Below is a structured, professional breakdown of the above two SQL scripts, (***techsphere_solutions and sample queries***) so you fully understand what each part is doing at a database design level.

A. TECHSPHERE_SOLUTIONS (Database and Tables)

1. Database Creation Section

```
DROP DATABASE IF EXISTS techsphere_solutions;
CREATE DATABASE techsphere_solutions;
USE techsphere_solutions;
```

What this does:

- **DROP DATABASE IF EXISTS**
  - Deletes the database if it already exists.
  - Prevents errors when rerunning the script multiple times.
- **CREATE DATABASE**
  - Creates a new database called techsphere_solutions.
- **USE**
  - Activates the database so all tables are created inside it.

Think of this as:
"Reset the system and start fresh in a new project workspace."

2. TABLE: departments

```
CREATE TABLE departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE,
    department_location VARCHAR(100)

);
```

Purpose:

Stores company departments (like IT, AI, Cybersecurity).
Columns:

- **department_id**
  - Unique identifier for each department
  - AUTO_INCREMENT → MySQL generates it automatically
- **department_name**

    - Name of department
    - NOT NULL → cannot be empty
    - UNIQUE → no duplicate departments allowed
- **department_location**
    - Where the department is located (e.g., Nairobi)

This is a **parent table** (other tables depend on it).

3. TABLE: employees

```
CREATE TABLE employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    gender ENUM('Male', 'Female', 'Other'),
    date_of_birth DATE,
    email VARCHAR(100) UNIQUE,
    phone_number VARCHAR(20),
    hire_date DATE NOT NULL,
    department_id INT,

    CONSTRAINT fk_department
        FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Purpose:

Stores employee personal and organizational data.

Key columns:

- **employee_id**
    - Primary key (unique employee identifier)
- **first_name / last_name**
    - Employee identity details
- **gender ENUM**
    - Restricts values to: Male, Female, Other
    - Ensures data consistency
- **date_of_birth**
    - Employee age tracking

- **email UNIQUE**
  - Prevents duplicate emails
- **hire_date**
  - When employee joined company
- **department_id**
  - Links employee to a department (relationship field)

Foreign Key Relationship

```
CONSTRAINT fk_department
FOREIGN KEY (department_id)
REFERENCES departments(department_id)
ON DELETE SET NULL
ON UPDATE CASCADE;
```

Meaning:

This connects employees → departments.

- **FOREIGN KEY**
  - department_id in employees must exist in departments table

Rules:

- **ON DELETE SET NULL**
  - If a department is deleted → employees remain but department becomes NULL
- **ON UPDATE CASCADE**
  - If department_id changes → automatically updates in employees

This ensures **data integrity**

4. TABLE: job_roles

```
CREATE TABLE job_roles (

   role_id INT AUTO_INCREMENT PRIMARY KEY,
   role_title VARCHAR(100) NOT NULL UNIQUE,
   base_salary DECIMAL(10,2) NOT NULL

);
```

Purpose:

Defines job positions in the company.

Columns:

- **role_id**
  - Unique ID for role
- **role_title**
  - Job title (e.g. Data Analyst)
- **base_salary**
  - Standard salary for that role

This is a **reference table** used for payroll logic.

5. TABLE: employee_positions (Junction Table)

```
CREATE TABLE Empoyee_positions (

    position_id INT AUTO_INCREMENT PRIMARYKEY,
    employee_id INT Not Null,
    role_id INT Not Null,
    start_date DATE Not Null,

    CONSTRAINT fk_employee
    FOREIGN KEY (employee_id)
    REFERENCES employees(employee_id)
    ON DELETE CASCADE

    CONSTRAINT fk_role
    FOREIGN KEY (role_id)
    REFERENCES job_roles(role_id)
    ON DELETE CASCADE
);
```

Purpose:

This is a **bridge table** connecting:

- Employees
- Job roles

Because:

- One employee can change roles over time
- One role can have many employees

So, this is a **Many-to-Many relationship resolver**

Foreign Keys:

Employee link

```
FOREIGN KEY (employee_id)
REFERENCES employees(employee_id)
ON DELETE CASCADE
```

- If employee is deleted → position record is also deleted

Role link

```
FOREIGN KEY (role_id)
REFERENCES job_roles(role_id)
ON DELETE CASCADE
```

- If role is deleted → related assignments are removed

6. INSERT INTO departments

```
INSERT INTO departments (department_name, department_location)
VALUES (...)
```

What happens:

You populate the departments table with real company-like units:

- Software Engineering
- Data Analytics
- Cybersecurity
- etc.

This creates the organizational structure.

7. INSERT INTO employees

```
INSERT INTO employees (…)
VALUES (...)
```

What happens:

Adds employees with:

- Personal info
- Department assignment

Example:

```
Benard → Data Analytics
Lilian → Software Engineering
Kevin → Cybersecurity
```

This creates **real-world workforce data**

8. INSERT INTO job_roles
Defines job categories:

- Data Analyst → 85,000
- AI Engineer → 160,000
- Cybersecurity Analyst → 135,000

These acts like a **salary benchmark table**

9. INSERT INTO employee_positions

```
INSERT INTO employee_positions
VALUES (employee_id, role_id, start_date)
```

What this does:

Assigns each employee a job role.
Example:

- Employee 1 → Data Analyst
- Employee 2 → Software Engineer

This connects the entire system together.

B. FEW SAMPLES OF SQL QUERIES

An SQL query is a structured command or instruction written in Structured Query Language (SQL) that is used to communicate with a database in order to retrieve, insert, update, or delete data. It represents a request that allows a user to interact with and manipulate information stored in a relational database.

1. View all employees

```
SELECT * FROM employees;
```

Meaning:

Fetches all employee records.

2. JOIN: Employees + Departments

```
SELECT
    e. employee_id,
    e.first_name,
    e.last_name,
    d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

What it does:

Combines:

- Employees table
- Departments table

LEFT JOIN meaning:

- Show ALL employees
- Even if department is missing

3. JOIN: Employees + Roles + Salary

```
SELECT
    e.first_name,
    e.last_name,
    jr.role_title,
    jr.base_salary
```

What it does:

Combines 3 tables:

- employees
- employee_positions
- job_roles

Result:

You see:

- Who the employee is
- Their job role
- Their salary

4. AGGREGATION: Average Salary by Department

```
SELECT
    d.department_name,
    AVG(jr.base_salary) AS average_salary
```

What it does:
Calculates:
Average salary per department
Key concept:

- AVG() = aggregation function
- GROUP BY = groups results per department

Final Output Meaning

This query answers:

"Which departments are the highest paying on average?"

BIG PICTURE (How the System Works)

Your database models a real company:

```
Departments (1)
↓
Employees (Many)
↓
Employee Positions (Bridge Table)
↓
Job Roles (Salary structure)
```